

LAPORAN PROJECT
PENGOLAHAN CITRA DIGITAL
SI SAINS DATA - UNIVERSITAS NEGERI SURABAYA

IDENTITAS PROYEK	
Judul	Perbandingan Model CNN dan Vision Transformer pada Sistem Optical Character Recognition untuk Konversi Citra Teks
Topik	Image To Conversion
Identitas Penyusun	1. Fitri Saidatur Hanifah (24031554035) 2. Yuli Inka Valentina Simanjuntak (24031554118) 3. Darista Wardhani (24031554146)
Kelas	2024D

1. PENDAHULUAN

1.1.Latar Belakang

Pengolahan citra digital (image processing) adalah sekumpulan metode dalam pemrosesan citra digital yang bertujuan untuk meningkatkan kualitas gambar atau mengekstraksi informasi yang berguna dari suatu citra [1]. Perkembangan teknologi kecerdasan buatan yang pesat dalam beberapa tahun terakhir telah mendorong berbagai penelitian di bidang pengenalan karakter secara otomatis, khususnya melalui pendekatan Optical Character Recognition (OCR) [2]. Pengenalan karakter alfabet merupakan salah satu permasalahan klasik dalam bidang visi komputer (computer vision) dan pengolahan citra. Sistem pengenalan karakter memiliki peran penting dalam berbagai aplikasi nyata, seperti digitalisasi dokumen, pembacaan otomatis formulir, sistem identifikasi, hingga teknologi bantu bagi penyandang disabilitas visual [3]. Meskipun telah banyak penelitian yang dilakukan, tantangan dalam pengenalan karakter masih tetap ada, terutama ketika karakter ditulis atau dicetak dalam berbagai jenis font yang memiliki gaya visual yang beragam [4].

Dataset Alphabet Characters Fonts Dataset yang tersedia di Kaggle memuat ribuan citra karakter alfabet A–Z dalam berbagai variasi font. Keberagaman font ini menjadi tantangan tersendiri karena setiap font memiliki karakteristik visual yang berbeda, seperti ketebalan garis, kemiringan, dan gaya huruf, sehingga model klasifikasi perlu mampu menggeneralisasi pola-pola tersebut secara akurat [5]. Convolutional Neural Network (CNN) telah terbukti menjadi pendekatan yang sangat efektif dalam tugas klasifikasi citra, termasuk dalam pengenalan karakter. Kemampuan CNN dalam mengekstraksi fitur secara hierarkis menjadikannya unggul dibandingkan metode konvensional seperti Support Vector Machine (SVM) dan Artificial Neural Network (ANN) biasa [6]. Penelitian oleh Christlein et al. (2017) menunjukkan bahwa CNN mampu mencapai akurasi hingga 98,8% dalam klasifikasi font pada dokumen teks [7]. Selain itu, teknik augmentasi data seperti rotasi, flipping, perubahan skala, dan

penambahan noise terbukti mampu meningkatkan kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya [8]. Preprocessing citra, termasuk konversi grayscale, normalisasi piksel, dan thresholding, juga merupakan langkah penting untuk memastikan kualitas input data sebelum dimasukkan ke dalam model [9]. Berdasarkan latar belakang tersebut, penelitian ini bertujuan untuk membangun sistem klasifikasi karakter alfabet berdasarkan variasi font menggunakan metode deep learning, khususnya CNN, dengan memanfaatkan Alphabet Characters Fonts Dataset dari Kaggle. Diharapkan sistem yang dibangun dapat mengklasifikasikan karakter alfabet dengan akurasi yang tinggi meskipun terdapat variasi font yang beragam [10].

1.2. Pendekatan Pemecahan Masalah

Permasalahan utama dalam penelitian ini adalah bagaimana membangun model klasifikasi citra yang mampu mengenali karakter alfabet (A–Z) dari berbagai jenis font secara akurat. Untuk mengatasi permasalahan tersebut, pendekatan yang digunakan adalah sebagai berikut:

2. Pengumpulan dan Eksplorasi Data, yaitu Dataset karakter alfabet dalam berbagai font diambil dari Kaggle (Alphabet Characters Fonts Dataset) dan dilakukan eksplorasi awal untuk memahami distribusi kelas dan karakteristik visual tiap karakter.
3. Preprocessing Citra, yaitu Citra diproses melalui tahapan konversi ke grayscale, normalisasi nilai piksel ke rentang $[0, 1]$, resize ke ukuran seragam, serta augmentasi data untuk memperkaya variasi data latih.
4. Perancangan dan Pelatihan Model CNN, yaitu Model CNN dirancang dengan beberapa lapisan konvolusi, pooling, dropout, dan fully connected layer untuk mengklasifikasikan 26 kelas karakter alfabet.
5. Evaluasi Model, yaitu Performa model dievaluasi menggunakan metrik akurasi, precision, recall, dan F1-score pada data uji untuk mengukur kemampuan generalisasi model

2. Metodologi

2.1 Eksplorasi Dataset

Dataset yang digunakan dalam penelitian ini adalah Alphabet Characters Fonts Dataset yang tersedia secara publik di platform Kaggle. Dataset ini memuat citra karakter alfabet A hingga Z yang dibangkitkan secara komputasional dari ratusan jenis font digital, dan disusun dalam struktur folder terpisah untuk masing-masing kelas karakter di dalam direktori Images/Images/. Mengingat ukuran dataset asli yang sangat besar, yakni sekitar 390.000 citra, dilakukan proses sampling untuk menjaga efisiensi komputasi selama eksperimen. Dari setiap kelas diambil sebanyak 1000 gambar secara acak, sehingga total dataset yang digunakan dalam penelitian ini berjumlah 26.000 citra yang terdistribusi merata ke dalam 26 kelas. Gambar hasil sampling disimpan ke dalam folder sampling-dataset/ dengan susunan subfolder per kelas yang konsisten. Sesudah dilakukan proses sampling, dilakukan eksplorasi data secara menyeluruh untuk memahami karakteristik dataset sebelum masuk ke tahap pelatihan model.

Eksplorasi ini mencakup tujuh aspek utama. Yang pertama, dilakukan distribusi jumlah gambar per kelas divisualisasikan menggunakan diagram batang untuk memverifikasi keseimbangan antar kelas. Kedua, dimana sampel gambar dari masing-masing kelas ditampilkan sebanyak tiga contoh guna memastikan kualitas visual citra setelah preprocessing. Ketiga, statistik nilai piksel dihitung untuk setiap kelas, meliputi nilai rata-rata, standar deviasi, minimum, dan maksimum. Keempat, visualisasi histogram distribusi nilai piksel dibuat baik secara global maupun per kelas untuk melihat sebaran intensitas cahaya pada citra. Kelima, rata-rata gambar per kelas (average image) dihasilkan untuk memperoleh gambaran representatif dari pola visual tiap karakter. Keenam, ukuran seluruh citra diverifikasi untuk memastikan keseragaman dimensi. Ketujuh, seluruh hasil eksplorasi dirangkum dalam satu tampilan ringkasan statistik yang mencakup jumlah kelas, total gambar, ukuran citra, format piksel, dan pipeline preprocessing yang diterapkan. Setelah akhir proses eksplorasi, diketahui bahwa dataset hasil preprocessing bersifat seimbang dengan 100 gambar per kelas, seluruh citra memiliki ukuran seragam 64x64 piksel dalam format grayscale, serta nilai piksel bersifat biner antara 0 dan 255 sebagai hasil dari proses thresholding Otsu yang diterapkan sebelumnya.

2.2 Langkah Penelitian

Tahapan penelitian Image to Conversion dengan dataset Alphabet Characters Fonts akan dilakukan secara sistematis dengan langkah langkah berikut.

Berdasarkan Gambar, berikut langkah-langkah penelitian yang akan dilakukan :

1. Pengumpulan Dataset

Dataset diperoleh dari platform kaggle yang terdiri dari berbagai karakter alfabet bervariasi bentuk font yang digunakan sebagai data pelatihan dan pengujian sistem. Dataset akan diimpor langsung dari platform menggunakan API resmi kaggle untuk mengurangi kendala RAM overload saat penelitian.

2. Sampling Dataset

Ukuran dataset yang digunakan sangat besar, proses sampling dilakukan untuk mengambil sebagian data dari setiap kelas karakter. Proses ini akan mengurangi penggunaan memori dan mempercepat proses penelitian yang akan dilakukan selanjutnya. Proses sampling akan mengambil citra secara acak dari setiap kelasnya.

3. Pre-processing Citra

Tahapan preprocessing dilakukan untuk meningkatkan kualitas citra dan mempersiapkan data sebelum modelling. Proses yang dilakukan berupa resize citra untuk menyeragamkan ukuran citra, konversi grayscale untuk mengurangi kompleksitas data, noise reduction untuk memperhalus citra, thresholding untuk mengubah citra menjadi hitam-putih, normalization agar proses pelatihan model menjadi lebih stabil.

4. Augmentasi Data

Augmentasi dilakukan untuk memperbanyak variasi data training secara otomatis agar mampu mengenali karakter dengan berbagai variasi bentuk dan posisi serta mengurangi risiko overfitting selama proses pelatihan. Teknik augmentasi yang dilakukan berupa rotasi, pergeseran posisi, zoom, dan flipping.

5. Pelatihan Model

Menyesuaikan dataset yang digunakan, metode yang akan digunakan yaitu Convolutional Neural Network (CNN). CNN dipilih karena memiliki kemampuan yang baik dalam mengenali pola visual dan melakukan klasifikasi citra.

6. Pengujian Sistem

Model yang dibangun akan diuji menggunakan data testing OCR yang disediakan. Sebelum dilakukan pengujian, data testing juga melalui tahap preprocessing agar memiliki format yang sama dengan data training. Tahap pengujian sistem akan mencoba mengenali karakter dari citra masukan dan mengubahnya menjadi teks berdasarkan hasil prediksi model CNN.

7. Konversi Citra ke Teks

Hasil prediksi karakter yang diperoleh dari model kemudian disusun menjadi bentuk teks. Tahapan ini merupakan inti topik project yaitu mengubah informasi visual berupa karakter pada citra menjadi data teks yang dapat dibaca dan digunakan lebih lanjut.

8. Evaluasi Hasil

Tahap evaluasi bertujuan untuk mengukur performa model dalam mengenali karakter. Hasil evaluasi digunakan untuk mengetahui tingkat keberhasilan sistem dalam melakukan konversi citra menjadi teks serta menganalisis performa model yang telah dibangun.

3. Hasil dan Analisis

3.1 Sampling dan Preprocessing

Dataset yang berhasil diunduh diambil sampling sebanyak 1000 gambar per kelas secara acak dan dilakukan preprocessing. Terdapat 3 tahapan utama yang diterapkan pada setiap gambar:

1. Konversi Ke Grayscale

Gambar dibaca dalam mode grayscale dengan tujuan menghemat memori dan komputasi secara signifikan dan model CNN dan ViT hanya bisa menerima input 1 channel warna.

2. Resize ke 64 x 64 pixel

Proses ini dilakukan untuk mengubah semua gambar 64 x 64 piksel secara seragam. Tujuan dari dilakukan Resize yaitu model deep learning membutuhkan input berukuran tetap sebagai keseimbangan antara detail karakter yang cukup dan efisiensi komputasi

3. Histogram Equalization

Tahapan ini dilakukan normalisasi kontras menggunakan teknik Histogram Equalization. Cara kerjanya adalah meratakan distribusi intensitas pixel sehingga

kontras gambar meningkat secara otomatis. Tujuan dari proses ini adalah Membantu model lebih mudah mengenali tepi dan bentuk karakter dan mengurangi pengaruh variasi pencahayaan antar gambar dari dataset yang berbeda font.

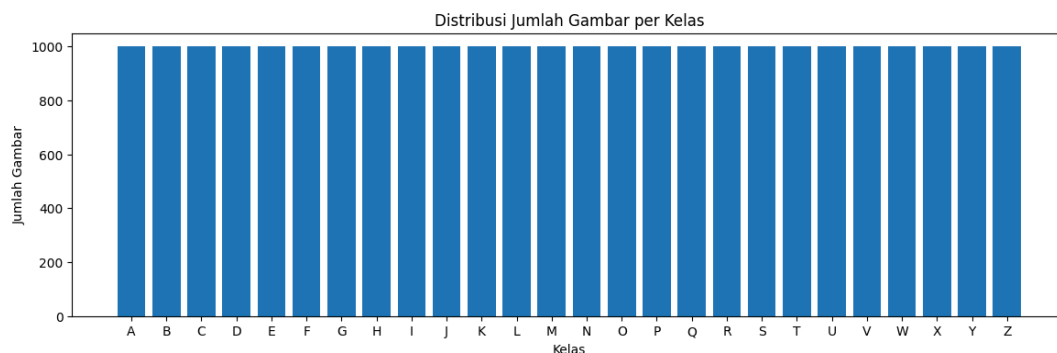
Proses ini dijalankan untuk seluruh kelas secara berurutan menggunakan tqdm sebagai progress bar, dan gambar hasil preprocessing disimpan kembali dengan nama file yang sama ke dalam folder preprocessed_dataset dengan struktur direktori per kelas yang identik dengan folder input.

3.2 Exploratory Data Analysis

Exploratory Data Analysis dilakukan setelah tahapan preprocessing dengan tujuan mengetahui informasi dataset yang siap dilakukan modelling setelahnya. Eksplorasi dataset dilakukan dengan beberapa proses berikut.

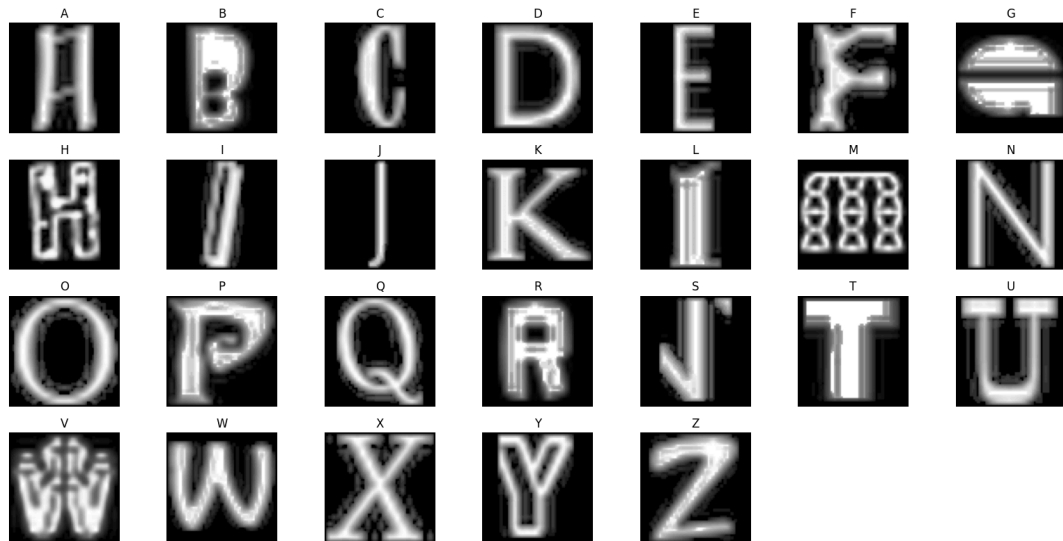
1. Distribusi Jumlah Gambar

Proses ini dijalankan untuk mengetahui distribusi jumlah gambar pada setiap kelas yang sudah dilakukan sampling sebelumnya. Dalam penelitian ini, digunakan 1000 gambar dengan jumlah 26 kelas yang mewakili huruf A-Z. Berdasarkan kode yang dijalankan, didapatkan informasi total gambar sebanyak 26.000 dengan nilai minimal dan maksimal gambar 1000.



2. Sample Gambar Per Kelas

Proses ini menampilkan sampel gambar perkelas secara acak dan ditampilkan dalam mode grayscale dengan judul nama kelasnya. Proses ini dilakukan dengan tujuan memastikan gambar sudah berhasil di preprocessing dengan benar dan memastikan tidak ada gambar yang korup dan salah kelas. Berikut visualisasi sampel gambar per kelas.

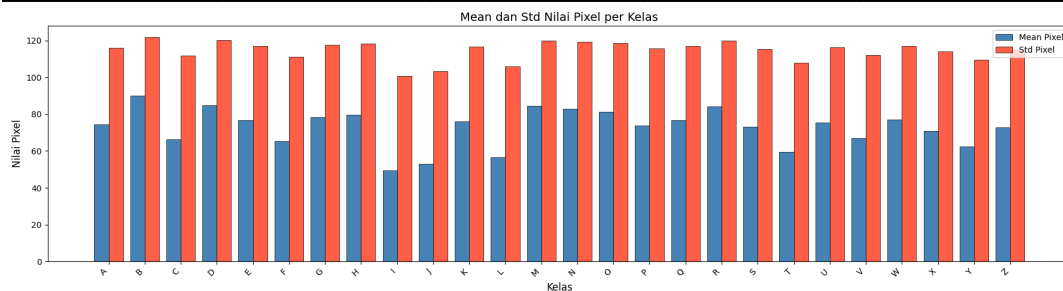


3. Statistik Deskriptif Nilai Pixel

Proses ini dilakukan untuk mengetahui nilai statistik pixel pada setiap kelas yang akan dilakukan modelling. Output dari proses ini dapat dilihat pada tabel (..)

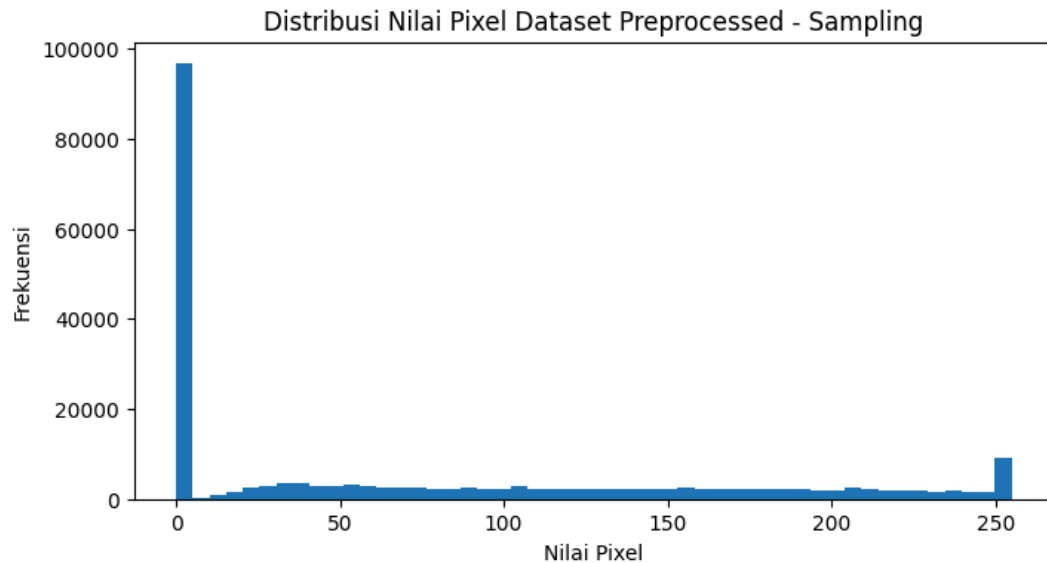
Kelas	Mean	Std	Min	Max
A	74.37	115.90	0	255
B	90.01	121.86	0	255
C	66.34	111.87	0	255
D	84.95	120.19	0	255
E	76.86	117.01	0	255
F	65.24	111.27	0	255
G	78.22	117.59	0	255
H	79.70	118.20	0	255
I	49.47	100.84	0	255
J	52.99	103.47	0	255
K	76.15	116.70	0	255
L	56.74	106.06	0	255
M	84.60	120.06	0	255
N	82.76	119.39	0	255
O	81.16	118.78	0	255

P	73.77	115.63	0	255
Q	76.70	116.94	0	255
R	84.08	119.88	0	255
S	73.04	115.29	0	255
T	59.57	107.90	0	255
U	75.38	116.36	0	255
V	67.13	112.30	0	255
W	77.08	117.11	0	255
X	70.78	114.19	0	255
Y	62.39	109.62	0	255
Z	72.75	115.15	0	255



4. Distribusi Nilai Pixel

Proses ini menjadi pelengkap informasi peneliti untuk mengetahui distribusi nilai pixel pada gambar yang digunakan dalam pelatihan model. Dalam proses ini masing-masing kelas diambil sampling 20 gambar secara acak dengan tujuan efisiensi memori. Dari 20 sampel gambar yang terpilih, hanya diambil 10 pixel pada setiap gambarnya. Tujuan dari proses ini adalah mengecek apakah ada dominasi nilai gelap (0) atau terang (255) yang ekstrem, yang bisa mengindikasikan masalah pada data.



5. Verifikasi Ukuran Gambar

Proses ini memeriksa dimensi (Height x Width) dari setiap gambar. Informasi yang didapatkan dari proses ini bahwa tahap preprocessing sebelumnya berjalan sempurna, dimana semua 58.785 gambar berhasil di-resize menjadi 64×64 piksel tanpa ada satupun yang lolos dengan ukuran berbeda.

3.3 Pembangunan Model

Penelitian ini dibangun dari dua model pengolahan citra, yaitu model CNN (..) dan ViT (..). Kedua model bekerja dengan cara yang berbeda untuk dibandingkan performanya dalam mengubah gambar menjadi teks.

A. Model CNN

Model CNN dibangun dengan berlapis 3 blok konvolusi yang masing-masing bloknnya memiliki dua Conv2D berurutan. Cara kerja model CNN yaitu dengan input gambar huruf asli lalu dilakukan augmentasi acak dengan rotasi 30°, menggesernya sejauh 4%, dan di zoom 5%. Dari proses augmentasi menghasilkan gambar asli dengan variasi huruf miring, bergeser, dan zoom. Gambar yang dilakukan augmentasi akan masuk ke dalam lapis Conv2D dimana proses ini akan memfilter gambar 3 x 3 yang bergeser pada tiap pixelnya. Hasilnya berupa gambar dengan 32 feature map (32 x 32 x 32) yang masing-masing menyorot tiap fitur yang berbeda seperti tepi horizontal, tepi vertikal, dan sudut/kurva. Berdasarkan hasil Feature map, dilakukan MaxPooling 2 x 2 dengan mempertahankan nilai terbesarnya untuk dijadikan output 2 x 2 pada 1 blok konvolusi. Proses Conv2D dilakukan sebanyak 3 blok (32 -> 64 -> 128), dimana semakin dalam filter maka gambar yang dihasilkan semakin abstrak. Output pada setiap blok dilakukan training dengan mematikan secara acak 25% output tersebut dengan tujuan mencegah model menghafal saat melakukan prediksi. Proses akhir adalah menentukan keputusan akhir dengan GlobalAveragePooling yang merangkum setiap Feature map 8×8 menjadi satu angka (8 x 8 x 128). Selanjutnya layer dense akan mencocokkan huruf yang dibaca dan softmax yang menentukan probabilitas untuk tiap 26 huruf. Berikut output yang dihasilkan dari model CNN.

Layer (type)	Output Shape	Parameters
input_layer (InputLayer)	(None, 64, 64, 1)	0
data_augmentation (Sequential)	(None, 64, 64, 1)	0
conv2d (Conv2D)	(None, 64, 64, 32)	320
batch_normalization (BatchNormalization)	(None, 64, 64, 32)	128
conv2d_1 (Conv2D)	(None, 64, 64, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 64, 64, 32)	128
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)	0
dropout (Dropout)	(None, 32, 32, 32)	0
conv2d_2 (Conv2D)	(None, 32, 32, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 32, 32, 64)	256
conv2d_3 (Conv2D)	(None, 32, 32, 64)	36,925
batch_normalization_3 (BatchNormalization)	(None, 32, 32, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)	0
dropout_1 (Dropout)	(None, 16, 16, 64)	0
conv2d_4 (Conv2D)	(None, 16, 16, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 16, 16, 128)	512
conv2d_5 (Conv2D)	(None, 16, 16, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 16, 16, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 128)	0
dropout_2 (Dropout)	(None, 8, 8, 128)	0
global_average_pooling2d	(None, 128)	0

(GlobalAveragePooling2D)		
dense (Dense)	(None, 256)	33,024
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 26)	6,682

B. Model ViT

Model ViT dibangun dengan PatchExtractor menggunakan `tf.image.extract_patches` untuk memotong gambar 64×64 menjadi 256 patch berukuran 4×4 pixel. Setiap patch di-flatten menjadi vektor 16 angka ($4 \times 4 \times 1$). Selanjutnya patch mentah 16 angka tersebut diberi dua perlakuan berbeda, perlakuan pertama yaitu proyeksi linear dimana angka 16 diubah menjadi 128 angka yang lebih bermakna dan perlakuan kedua yaitu positional embedding yang menambahkan informasi urutan patch agar model tahu posisinya dalam gambar. Setiap patch akan mencocokkan antar patch lainnya untuk mendapatkan skor korelasi, semakin relevan patch tersebut memahami lawan patch nya maka semakin tinggi skor korelasinya. Hasilnya setiap patch diperbarui dengan informasi dari seluruh gambar sekaligus. Selanjutnya Setiap Transformer Block yang diulang sebanyak 6 kali berisi dua sub-blok dengan residual connection. Pada sub blok pertama terdiri dari LayerNorm, MultiHeadAttention, dan menambahkan input asli. Sedangkan pada sub blok kedua terdiri dari LayerNorm, MLP [256→128] dengan aktivasi GELU, dan menambahkan input sub-blok pertama. Residual connection memastikan gradien bisa mengalir langsung ke layer awal tanpa hilang selama backpropagation.

3.4 Evaluasi Model

1. Evaluasi Model Data Test

Pertama, label asli (`y_true`) diambil dari kolom `label_encoded` pada dataframe test. Kemudian model melakukan prediksi terhadap seluruh gambar pada `test_ds` menggunakan `model.predict()`, yang menghasilkan probabilitas untuk setiap kelas (26 kelas A–Z). Kelas dengan probabilitas tertinggi dipilih sebagai hasil prediksi menggunakan `np.argmax()`. Setelah itu, `model.evaluate()` dijalankan untuk mendapatkan dua nilai utama yaitu Test Loss (seberapa besar kesalahan model secara numerik menggunakan fungsi Sparse Categorical Crossentropy) dan Test Accuracy (persentase prediksi yang benar dari seluruh data test). Selanjutnya dicetak Classification Report dari scikit-learn yang memperlihatkan metrik per kelas, yaitu:

- Precision: dari semua prediksi kelas X, berapa persen yang memang benar kelas X
- Recall: dari semua gambar kelas X, berapa persen yang berhasil dikenali
- F1-Score: rata-rata harmonik precision dan recall, memberikan gambaran keseimbangan keduanya
- Support: jumlah sampel asli per kelas di data test

Terakhir ditampilkan Confusion Matrix berukuran 26×26 dalam bentuk heatmap. Nilai tinggi pada diagonal utama berarti prediksi benar, sedangkan nilai di luar diagonal menunjukkan jenis kesalahan misklasifikasi antar kelas. Fungsi ini dijalankan dua kali, sekali untuk model CNN terbaik (`best_cnn_model.keras`) dan sekali untuk model ViT terbaik (`best_vit_model.keras`), keduanya dimuat dari Google Drive. Hasil kedua model dikumpulkan ke dalam `model_comparison_df` dan ditampilkan sebagai tabel serta bar chart perbandingan Test Accuracy. Evaluasi dilakukan pada 3.900 data test (150 gambar per kelas × 26 kelas) menggunakan model CNN dan ViT terbaik yang dimuat dari Google Drive. Test Loss: 0.5330 dan Test Accuracy: 0.8726 (87.26%). Artinya dari 3.900 gambar test, model CNN berhasil mengklasifikasikan dengan benar sekitar 3.403 gambar. Berikut hasil Classification Report CNN per kelas:

Hasil CNN :

Kelas dengan performa terbaik adalah S (F1=0.92), F, H, J, T, X, Y (F1=0.91). Kelas dengan performa terendah adalah O dan V (F1=0.80), serta G dan P (F1=0.82), yang kemungkinan sering tertukar satu sama lain karena kemiripan bentuk antar font.

Kelas	Precision	Recall	F1-Score	Support
A	0.93	0.87	0.90	150
B	0.90	0.86	0.88	150
C	0.88	0.82	0.85	150
D	0.89	0.79	0.84	150
E	0.85	0.89	0.87	150
F	0.90	0.92	0.91	150
G	0.78	0.87	0.82	150
H	0.90	0.92	0.91	150
I	0.85	0.90	0.87	150
J	0.95	0.87	0.91	150

K	0.90	0.87	0.88	150
L	0.91	0.86	0.89	150
M	0.93	0.84	0.88	150
N	0.87	0.82	0.84	150
O	0.75	0.86	0.80	150
P	0.75	0.91	0.82	150
Q	0.93	0.86	0.89	150
R	0.89	0.87	0.88	150
S	0.91	0.90	0.90	150
T	0.92	0.89	0.91	150
U	0.86	0.87	0.86	150
V	0.81	0.79	0.80	150
W	0.84	0.88	0.86	150
X	0.91	0.90	0.90	150
Y	0.91	0.91	0.91	150
Z	0.89	0.89	0.89	150
Macro Avg	0.88	0.87	0.87	3900
Weighted Avg	0.88	0.87	0.87	3900

Hasil ViT:

Test Loss: 3.2581 dan Test Accuracy: 0.0444 (4.44%) sangat buruk. Dari classification report terlihat hampir semua kelas memiliki precision, recall, dan F1-score 0.00, kecuali kelas T yang recall-nya 0.99 namun precision hanya 0.04. Ini menunjukkan model ViT hampir seluruh prediksinya jatuh ke kelas T saja (collapsed prediction), yang merupakan tanda model gagal belajar dengan benar kemungkinan besar karena model ViT tidak sempat ditraining cukup (checkpoint tidak tersedia atau training terpotong) sehingga bobot yang dimuat adalah model awal yang belum optimal.

Model	Test Loss	Test Accuracy
-------	-----------	---------------

CNN	0.5330	87.26%
ViT	3.2581	4.44%

2. Evaluasi Dengan Generate OCR

Daftar teks yang digunakan terdiri dari 15 kata berbahasa Inggris yang hanya mengandung huruf kapital A–Z (sesuai dengan kelas yang dilatih model), yaitu: DATA, SCIENCE, IMAGE, TEXT, PYTHON, MODEL, VISION, LETTER, CLASS, TRAIN, DEEP, LEARNING, NEURAL, NETWORK, OPTICAL. Setiap gambar dibuat dengan variasi berikut agar menyerupai kondisi nyata:

- Background: warna abu-abu muda hingga putih (nilai piksel 230–255) dipilih secara acak
- Font: dipilih secara acak dari font TrueType yang tersedia di sistem (DejaVu Sans, DejaVu Serif, Liberation Sans), dengan fallback ke font default PIL jika tidak ada font yang tersedia
- Ukuran font: acak antara 48–70 piksel
- Posisi teks: digeser sedikit secara acak (± 15 piksel horizontal, ± 10 piksel vertikal) agar tidak selalu di tengah persis
- Warna teks: hitam hingga abu-abu gelap (nilai piksel 0–60)
- Rotasi: miring kecil secara acak antara -3° hingga $+3^\circ$
- Noise: ditambahkan noise Gaussian dengan standar deviasi acak 3–10 piksel untuk mensimulasikan gambar yang tidak sempurna
- Blur: 40% gambar secara acak dikenai Gaussian Blur ringan (radius 0.3–0.8) untuk mensimulasikan gambar sedikit buram

3. Input Gambar OCR Dan Ground Thruth

Selain dataset semi-realistis yang di-generate otomatis, proyek ini juga menggunakan gambar nyata yang diunduh dari Google Drive sebagai bahan uji OCR yang lebih realistis. Proses pada bagian ini: Gambar diunduh dari folder Google Drive menggunakan `gdown.download_folder()` ke dalam folder lokal `ocr_input_images/`. Setelah download selesai, sistem melakukan scan rekursif ke semua subfolder untuk mengumpulkan semua file gambar yang ditemukan (format: .png, .jpg, .jpeg, .bmp, .tif, .tiff, .webp), kemudian disimpan sebagai daftar path gambar `ocr_image_paths` yang sudah diurutkan. Berdasarkan kode yang ada, terdapat 5 gambar OCR nyata yang digunakan sebagai data uji dengan ground truth yang telah diisi secara manual, yaitu:

Index	Ground Truth
0	CLICK TO ADD TITLE CLICK TO ADD

	SUBTITLE
1	263S2V
2	DSJCBKA
3	AAXUE
4	BK368H

4. Segmentasi, Prediksi dan Post Process OCR

File	Karakter GT	Objek Tersegmentasi	Selisih
ocr1.jpg	33	18	-15 (under)
ocr2.jpg	6	7	+1 (over)
ocr3.jpg	7	8	+1 (over)
ocr4.jpg	5	2	-3 (under)
ocr5.jpg	6	37	+31 (over ekstrem)

Langkah 1 – Deteksi Tipe Gambar

Fungsi `detect_image_type()` menganalisis gambar menggunakan ruang warna HSV dan grayscale. Untuk `ocr1.jpg` terdeteksi sebagai `colored_text` karena memiliki saturasi tinggi ($\text{mean saturation} > 30$ dan $\text{high_sat_pixels} > 5\%$), menandakan ada elemen berwarna dalam gambar — sesuai dengan template slide yang biasanya memiliki teks berwarna atau elemen desain. Untuk `ocr4.jpg` terdeteksi sebagai `noisy_binary` karena distribusi pikselnya sangat bimodal ($\text{rasio piksel gelap} + \text{piksel terang} > 70\%$), menandakan gambar sudah hampir biner hitam-putih.

Langkah 2 – Pemilihan Strategi Segmentasi

Berdasarkan tipe yang terdeteksi, fungsi segmentasi dipilih otomatis:

- `ocr1.jpg` → `segment_colored_text()`: bekerja di ruang warna HSV, menggabungkan mask dari kanal saturasi (piksel dengan saturasi > 60 dianggap teks berwarna) dan mask dari kanal value (piksel gelap dianggap teks hitam) menggunakan operasi OR. Kemudian dilakukan morfologi OPEN untuk menghapus noise kecil dan CLOSE untuk menutup celah dalam karakter.

- ocr4.jpg → `segment_noisy_binary()`: menggunakan threshold biasa (bukan Otsu) pada nilai 127, lalu morfologi CLOSE (kernel 5×5) untuk menutup celah dan OPEN (kernel 3×3) untuk menghapus noise.

Langkah 3 – Pembersihan Connected Components

Setiap komponen terhubung pada mask biner difilter menggunakan `clean_connected_components()` dengan empat kriteria sekaligus: area minimum (menghapus noise titik kecil), tinggi minimum (menghapus garis horizontal tipis yang bukan karakter), area maksimum (menghapus background yang ikut tersegmentasi), dan aspect ratio antara 0.05–5 (menghapus objek yang terlalu sempit atau terlalu lebar untuk menjadi karakter).

ocr1.jpg hanya menghasilkan 18 dari 33 karakter karena Template slide PowerPoint memiliki teks dengan opacity atau warna yang mungkin tidak konsisten. Beberapa karakter mungkin terlalu tipis atau kontrasnya rendah sehingga difilter oleh area minimum. Selain itu huruf yang berdekatan bisa bergabung menjadi satu kontur karena operasi morfologi CLOSE.

ocr4.jpg hanya menghasilkan 2 dari 5 karakter karena Pada gambar `noisy_binary`, threshold 127 mungkin tidak cocok dengan kecerahan gambar sebenarnya, sehingga karakter yang seharusnya gelap justru tidak tertangkap atau bergabung menjadi blok besar yang kemudian difilter oleh area maksimum.

ocr5.jpg menghasilkan 37 dari hanya 6 karakter karena Gambar BK368H kemungkinan memiliki tekstur atau noise yang kuat. Noise dan detail gambar ikut terbaca sebagai kontur terpisah karena ambang batas segmentasi tidak mampu membedakan antara karakter asli dan artefak gambar.

Langkah 4 – Deteksi dan Pemotongan Karakter

`cv2.findContours()` menemukan kontur pada mask bersih. Setiap kontur diubah menjadi bounding box (`x, y, w, h`) menggunakan `cv2.boundingRect()`. Padding 4 piksel ditambahkan di setiap sisi agar karakter tidak terpotong persis di tepinya. Setiap ROI kemudian diinversi ($255 - \text{nilai piksel}$) sehingga karakter menjadi hitam di background putih, lalu di-resize ke 64×64 dan dinormalisasi ke $[0,1]$.

Langkah 5 – Reading Order

Boundingbox dikelompokkan ke baris-baris menggunakan `group_boxes_into_lines()`. Threshold jarak antar baris = 60% dari median tinggi semua bounding box. Dalam setiap baris, box diurutkan berdasarkan posisi X dari kiri ke kanan.

Langkah 6 – Prediksi dan Post-Process

Model CNN memprediksi setiap potongan karakter sekaligus dalam satu batch. Untuk ocr1.jpg yang memiliki 18 potongan, model menghasilkan 18 prediksi. Deteksi spasi dilakukan dengan membandingkan gap antar bounding box terhadap $1.5 \times$ lebar median karakter — jika gap lebih besar, sisipkan spasi. Karakter dengan confidence < 0.20 diganti dengan tanda tanya "?". Inilah mengapa output ocr1.jpg mengandung tanda "?" pada beberapa posisi (C?GC).

5. Evaluasi OCR

File	Ground Truth	Prediksi	CER	WER	Char Accuracy	Word Accuracy	Exact Match
ocr1.jpg	CLICK TO ADD TITLE CLICK TO ADD SUBTITLE	ZC C C C C?GC EO O G CC BC C	0.7250	1.375	0.275	0	0
ocr2.jpg	263S2V	ZZ Z RZWW	1.5000	3.000	0.000	0	0
ocr3.jpg	DSJCB KA	ZZWG ZCC Z	1.1429	2.000	0.000	0	0
ocr4.jpg	AAXU E	CC	1.0000	1.000	0.000	0	0
ocr5.jpg	BK368 H	ZZ?IK? ZD? C??IZ? ZI?CZ? CCIZ?? ? C ZZ?II? Z?	6.5000	4.000	0.000	0	0

- ocr1.jpg — CER 0.7250, Char Accuracy 0.275

Ground truth memiliki 40 karakter termasuk spasi, atau 33 karakter tanpa spasi. Setelah normalisasi (konversi uppercase, hapus spasi ganda), Levenshtein Distance antara prediksi dan ground truth adalah 29. $CER = 29/40 = 0.725$. Ini adalah yang paling rendah (paling baik) di antara semua gambar, karena beberapa karakter kebetulan terprediksi benar atau hampir benar. $WER = 1.375$ berarti dibutuhkan 11

operasi edit pada level kata, dan karena ada 8 kata dalam ground truth, $WER = 11/8 = 1.375$ (lebih dari 1 karena prediksi menghasilkan lebih banyak "kata" dari jumlah aslinya).

- ocr2.jpg — CER 1.5000, WER 3.000

Ground truth "263S2V" memiliki 6 karakter. Prediksi "ZZ Z RZWW" memiliki 9 karakter (termasuk spasi). Levenshtein Distance = 9. $CER = 9/6 = 1.5$ (lebih dari 1 karena prediksi lebih panjang dari ground truth). Angka 2, 6, 3 tidak bisa dikenali sama sekali karena tidak ada dalam kelas training. $WER = 3.0$ berarti dari 1 kata ground truth, dibutuhkan 3 operasi edit kata, karena prediksi terpecah menjadi beberapa "kata" akibat spasi yang salah.

- ocr3.jpg — CER 1.1429, WER 2.000

Ground truth "DSJCBKA" memiliki 7 karakter. Prediksi "ZZWGZCC Z" memiliki 9 karakter. Levenshtein Distance = 8. $CER = 8/7 = 1.1429$. $WER = 2/1 = 2.0$ karena prediksi terpecah menjadi 2 kata (ada spasi di tengah) sedangkan ground truth hanya 1 kata.

- ocr4.jpg — CER 1.000

Ground truth "AAXUE" memiliki 5 karakter. Prediksi hanya "CC" (2 karakter). Levenshtein Distance = 5 (semua karakter harus diganti atau diinsert). $CER = 5/5 = 1.0$. Ini adalah kasus under-segmentasi parah di mana hanya 2 objek yang terdeteksi dari 5 karakter.

- ocr5.jpg — CER 6.500, WER 4.000 (terburuk)

Ground truth "BK368H" memiliki 6 karakter. Prediksi "ZZ?IK?ZD? C??IZ?ZI?CZ?CCIZ??? C ZZ?II?Z?" memiliki 39 karakter aktif (termasuk tanda tanya). Levenshtein Distance = 39. $CER = 39/6 = 6.5$ — jauh melebihi 1.0 karena prediksi sangat jauh lebih panjang dari ground truth akibat 37 objek yang tersegmentasi (over-segmentasi ekstrem). Banyaknya tanda tanya menunjukkan banyak potongan noise yang confidence-nya di bawah 0.20.

6. Ringkasan Evaluasi OCR

===== RINGKASAN EVALUASI OCR =====

Jumlah data : 5
Exact Match Accuracy : 0.0000
Mean CER : 2.1736
Mean WER : 2.2750
Mean Character Accuracy : 0.0550
Mean Word Accuracy : 0.0000

- Exact Match Accuracy = 0.0000 dihitung sebagai rata-rata kolom `exact_match` dari semua baris. Karena semua 5 gambar menghasilkan `exact_match` = 0, rata-ratanya adalah $0/5 = 0.0$. Tidak satu pun gambar yang prediksinya persis sama dengan ground truth.
- Mean CER = 2.1736 adalah rata-rata dari $(0.725 + 1.500 + 1.143 + 1.000 + 6.500) / 5 = 10.868 / 5 = 2.1736$. Nilai ini jauh di atas 1.0 yang sudah dianggap sangat buruk. Nilai rata-rata ditarik sangat tinggi oleh `ocr5.jpg` yang memiliki CER 6.5 akibat over-segmentasi ekstrem.
- Mean WER = 2.2750 adalah rata-rata dari $(1.375 + 3.000 + 2.000 + 1.000 + 4.000) / 5 = 11.375 / 5 = 2.275$. Nilai WER di atas 1.0 terjadi karena pipeline OCR memecah teks menjadi lebih banyak "kata" dari seharusnya akibat spasi yang salah disisipkan, atau sebaliknya menghasilkan teks jauh lebih panjang (`ocr5.jpg`).
- Mean Character Accuracy = 0.0550 (5.5%) adalah rata-rata dari $(0.275 + 0 + 0 + 0 + 0) / 5 = 0.275 / 5 = 0.055$. Hanya `ocr1.jpg` yang memberikan kontribusi nilai positif ($0.275 = 27.5\%$), sementara empat gambar lainnya menghasilkan Char Accuracy 0. Secara rata-rata, dari seluruh karakter yang ada, pipeline hanya mampu "mendekati benar" sebesar 5.5% saja.
- Mean Word Accuracy = 0.0000 karena WER semua gambar ≥ 1.0 , sehingga Word Accuracy = $\max(0, 1 - \text{WER}) = 0$ untuk semua gambar.
- Analisis Kesalahan (Error Analysis): Karena semua 5 gambar menghasilkan `exact_match` = 0, semua 5 gambar masuk ke dalam tabel analisis kesalahan. Ini mengkonfirmasi bahwa pipeline OCR yang dibangun belum mampu bekerja pada gambar dunia nyata dengan kondisi yang beragam. Akar masalahnya ada di dua lapis: tahap segmentasi yang menghasilkan jumlah potongan karakter yang salah (terlalu sedikit atau terlalu banyak), dan tahap klasifikasi yang tidak mengenali karakter hasil segmentasi karena distribusinya berbeda dari data training. Perbaikan yang diperlukan adalah menggunakan model yang dilatih khusus pada data OCR nyata, atau menerapkan teknik domain adaptation agar model mampu menangani variasi kondisi gambar di dunia nyata.

6. Daftar Pustaka

- [1] Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson Education.
- [2] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- [3] Kumar, M., & Sharma, R. K. (2016). Optical character recognition: A review. *International Journal of Advanced Research in Computer Science and Software Engineering*, 6(3), 1–7.
- [4] Wang, K., Babenko, B., & Belongie, S. (2015). End-to-end scene text recognition. *IEEE International Conference on Computer Vision (ICCV)*, 1457–1464.
- [5] Thomasqazwsxedc. (2023). Alphabet Characters Fonts Dataset. Kaggle. <https://www.kaggle.com/datasets/thomasqazwsxedc/alphabet-characters-fonts-dataset>
- [6] Suartika, I. W. E. P., Wijaya, A. Y., & Soelaiman, R. (2016). Klasifikasi citra menggunakan Convolutional Neural Network (CNN) pada Caltech 101. *Jurnal Teknik ITS*, 5(1), A65–A69.

- [7] Christlein, V., Bernecker, D., Hönig, F., Maier, A., & Angelopoulou, E. (2017). Convolutional neural networks for font classification. Proceedings of ICDAR, 2017. <https://arxiv.org/abs/1708.03669>
- [8] Bora, M. B., Daimary, D., Amitab, K., & Kandar, D. (2020). Handwritten character recognition from images using CNN-ECOC. Procedia Computer Science, 167, 2403–2409. <https://doi.org/10.1016/j.procs.2020.03.293>
- [9] Hidayatullah, P. (2017). Pengolahan Citra Digital: Teori dan Aplikasinya. Informatika Bandung.
- [10] Ahmed, R. M., Rashid, T. A., & Fattah, P. (2022). Kurdish handwritten character recognition using deep learning techniques. arXiv preprint, arXiv:2210.13734. <https://arxiv.org/abs/2210.13734>